

Analiza Experimentală și Complexitatea Asimptotică a Algoritmilor de Sortare

Proiect de Cercetare - Scriere Academică

Octavian-Mario Inoveanu

Facultatea de Informatică(UVT)

30 aprilie 2026

Rezumat

Prezenta lucrare expune o analiză empirică și teoretică a metodelor fundamentale de sortare. Prin evaluarea algoritmilor cu limită pătratică, logaritmică și liniară pe seturi de date de $N = 50$ și $N = 50.000$ de elemente, studiul evidențiază discrepanțele dintre complexitatea asimptotică $O(f(n))$ și latența reală datorată arhitecturii hardware moderne (ierarhii de memorie cache, predicția ramificațiilor).

Cuvinte cheie: sortare, complexitate computațională, recurență, $O(N \log N)$, ierarhie cache.

1 Introducere

Sortarea reprezintă una dintre cele mai riguros analizate probleme din domeniul științei calculatoarelor. În ciuda existenței unor limitări matematice clare privind timpul minim de execuție, comportamentul algoritmilor variază masiv în funcție de topologia datelor de intrare.

State of the art: La momentul actual, implementările standard din industrie (cum ar fi `std::sort` din C++ sau `TimSort` din Python) nu se bazează pe un singur algoritm, ci pe metode hibride. Acestea folosesc algoritmi de partiționare ($O(N \log N)$) pentru volume mari, dar comută pe algoritmi de inserție ($O(N^2)$) când sub-problemele devin suficient de mici pentru a încăpea în memoria L1 Cache a procesorului.

2 Analiza problemei

Descrierea formală a problemei sortării se definește astfel: dată fiind o secvență de N chei $A = \langle a_1, a_2, \dots, a_n \rangle$, se cere determinarea unei permutări $\pi : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$ astfel încât secvența rezultată $A' = \langle a_{\pi(1)}, a_{\pi(2)}, \dots, a_{\pi(n)} \rangle$ să respecte relația de ordine:

$$a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$$

Pentru a demonstra performanța teoretică, algoritmi bazați pe comparare sunt mărginiți inferior de un arbore de decizie, stabilind limita teoretică la $\Omega(N \log N)$.

În cadrul lucrării analizăm comportamente specifice:

- **Merge Sort (Divide et Impera):** Descompune secvența în jumătăți. Timpul de execuție poate fi modelat prin ecuația de recurență:

$$T(n) = \begin{cases} \Theta(1) & \text{dacă } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{dacă } n > 1 \end{cases}$$

Conform Teoremei Master, soluția acestei recurențe este $T(n) = \Theta(N \log N)$.

- **Insertion Sort:** Deși aparține clasei $O(N^2)$, pentru o secvență *aproape sortată* (numărul de inversiuni este limitat la $k \ll N$), complexitatea sa degradează într-o funcție liniară $O(N + k)$, devenind soluția optimă teoretică.
- **Bubble Sort:** Se bazează pe interschimbări repetate ale elementelor adiacente. Timpul de execuție în cazul mediu și defavorabil este pătratic, însă poate fi optimizat la $O(N)$ în cel mai bun caz (listă deja sortată) prin introducerea unui indicator de oprire timpurie. Numărul maxim de comparații este:

$$T(N) = \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2} = \Theta(N^2)$$

- **Selection Sort:** Împarte lista într-o sublistă sortată și una nesortată, căutând iterativ minimumul. Complexitatea este independentă de entropia inițială a datelor, fiind $O(N^2)$ în absolut toate cazurile, deoarece necesită mereu calculul exhaustiv al minimumului rămas.
- **Cycle Sort:** Reprezintă algoritmul optim din punct de vedere al numărului de scrieri în memorie (fiecare element este scris la poziția sa finală cel mult o singură dată). Deși complexitatea temporală rămâne $\Theta(N^2)$, este superior pe arhitecturi hardware unde operațiile de scriere sunt costisitoare sau uzează mediul de stocare.
- **Heap Sort:** Transformă secvența într-o structură arborescentă de tip *Max-Heap*. Construirea inițială a arborelui (*heapify*) durează $O(N)$, iar cele N extrageri succesive ale maximumului costă fiecare $O(\log N)$. Astfel, complexitatea globală este asimptotic optimă pentru o sortare prin comparație:

$$T(N) = O(N) + \sum_{i=1}^N O(\log i) = \Theta(N \log N)$$

Un avantaj teoretic major este complexitatea spațială auxiliară de $O(1)$, algoritmul rulând strict *in-place*.

- **Radix Sort:** Este o metodă de sortare non-comparativă ce procesează cheile cifră cu cifră (sau bit cu bit), de la cea mai puțin semnificativă la cea mai semnificativă. Astfel, evită limitarea inferioară de $\Omega(N \log N)$. Timpul de execuție este $O(N \cdot K)$, unde K reprezintă numărul maxim de cifre. Pentru chei întregi limitate, se comportă ca o sortare în timp liniar.

3 Modelarea problemei și implementarea soluției

Modelarea soluției s-a realizat în limbajul C++17, folosind paradigma *Orientată pe Obiecte (OOP)* pentru modularizarea codului. Algoritmii sunt încapsulați conform design pattern-ului *Strategy*, asigurând o interfață comună de testare.

Structurile de date primare utilizate sunt vectorii alocați contiguu în memorie (`std::vector`). Această alegere nu este întâmplătoare; alocarea contiguă reduce dramatic fenomenul de *cache miss*, oferind un avantaj practic algoritmilor cu acces liniar (Merge Sort) față de cei cu acces arborescent non-liniar (Heap Sort).

Pentru a gestiona eficient execuția algoritmilor și a implementa un mecanism de control al timpului, arhitectura aplicației a explorat paradigma de *procesare concurentă* (multithreading), utilizând facilitățile standardului modern C++:

- **Lansarea asincronă (`std::async`):** Fiecare algoritm de sortare a fost delegat către un fir de execuție (thread) separat. Obiectele de tip `std::future` au fost folosite pentru a monitoriza stadiul fiecărui task, permițând aplicației să aștepte finalizarea tuturor rutinelor.
- **Sincronizarea resurselor (`std::mutex`):** Pentru a preveni fenomenul de *data race* (coruperea datelor la scrierea concurentă), salvarea metricilor în lista globală de rezultate și afișarea mesajelor în consolă au fost protejate prin secțiuni critice, controlate cu `std::lock_guard<std::mutex>`.
- **Mecanismul de Timeout (`std::atomic`):** Deoarece algoritmii cu limită pătratică $O(N^2)$ pot dura un timp nerezonabil de lung pe seturi mari de date ($N = 1.000.000$), s-a implementat o variabilă thread-safe `std::atomic<bool> stop_flag`. Un thread principal ("watchdog") oprește forțat execuția tuturor algoritmilor aflați în rulare dacă se depășește pragul de timp alocat.

Deși această arhitectură este robustă, este important de menționat că rularea masiv-paralelă a algoritmilor generează *contention* (competiție) pe memoria cache a procesorului. Astfel, pentru acuratețea științifică a benchmark-ului, măsurătorile finale de performanță (timp și RAM) sunt înregistrate forțând o execuție secvențială și izolată.

Generarea datelor s-a realizat printr-un sistem robust bazat pe `std::mt19937` (Mersenne Twister), evitând defectele statistice ale clasicului `rand()` din C.

Timpul a fost monitorizat folosind ceasul de înaltă rezoluție:

$$\Delta t = T_{final} - T_{start} \quad (\text{măsurat în nanosecunde})$$

4 Studii de caz (experiment)

Conform metodologiei de cercetare, studiul de caz constă în prezentarea comportamentului implementării în practică, incluzând modelele datelor, măsurarea și interpretarea rezultatelor. Experimentul a fost proiectat pentru a evidenția comportamentul algoritmilor la două scări diametral opuse:

- **Liste mici ($N = 50$):** Pentru a observa impactul costurilor constante (overhead-ul apelurilor recursive și alocărilor dinamice de memorie).
- **Liste mari ($N = 50.000$):** Pentru a valida ratele de creștere asimptotică și a forța algoritmii în regim de stres computațional.

Pentru ambele dimensiuni N , s-au generat câte 4 topologii de date distincte:

1. **Complet aleatoare:** Numere generate folosind o distribuție uniformă, reprezentând cazul mediu.
2. **Sortate crescător:** Modelează cel mai favorabil caz teoretic (*best-case scenario*) pentru unii algoritmi bazați pe inserție.
3. **Sortate descrescător:** Modelează cel mai defavorabil caz teoretic (*worst-case scenario*), maximizând numărul de interschimbări.
4. **Aproape sortate:** O permutare inițial ordonată în care 5% dintre elemente au fost interschimbate.

La scară mică ($N = 50$), s-a observat că algoritmi elementari (precum *Insertion Sort*) depășesc frecvent metodele avansate. Acest fenomen se explică prin absența stivei de apeluri recursive pe care o necesită *Merge Sort* sau *Heap Sort*.

La $N = 50.000$, discrepanța devine colosală. Algoritmi cu limită $O(N^2)$ (*Bubble, Selection*) înregistrează timpi de ordinul sutelor de milisecunde, în timp ce algoritmi $O(N \log N)$ finalizează sortarea în sub 5 milisecunde, validând astfel dominația asimptotică.

5 Comparația cu literatura

Această secțiune plasează rezultatele obținute în contextul literaturii de specialitate, evaluând relevanța și comparând avantajele și dezavantajele.

Rezultatele experimentale se aliniază puternic cu postulatele teoretice (ex. Thomas Cormen - *Introduction to Algorithms*). Totuși, experimentul aduce o perspectivă practică asupra limitărilor hardware: deși *Heap Sort* are un profil de memorie mai bun $O(1)$ comparativ cu *Merge Sort* $O(N)$, accesul său non-liniar la structura de arbore implementată pe vector determină un număr masiv de *cache misses*. Astfel, literatura modernă confirmă de ce *Merge Sort* și *Radix Sort* (pe numere cu puține cifre) au timpi fizici de execuție mai buni pe arhitecturile procesoarelor actuale.

În cazul *Insertion Sort*, teoria indică o limită asimptotică de $O(N)$ pentru seturi aproape sortate. Experimentul pe datele de tipul 4 („Aproape sortate”) confirmă excelent acest lucru, algoritmul rulând mai rapid decât *Merge Sort* pe $N = 50.000$.

6 Concluzii și direcții viitoare

Această secțiune sintetizează rezultatele, detaliază dificultățile întâmpinate și oferă o perspectivă asupra direcțiilor viitoare de cercetare.

În concluzie, nu există un algoritm de sortare universal superior. Eficiența este o funcție care depinde de dimensiunea setului ($N = 50$ vs $N = 50.000$) și de entropia distribuției inițiale a elementelor.

Dificultăți întâmpinate: Pe parcursul implementării, principala dificultate a fost legată de izolarea măsurătorilor de memorie RAM. Încercarea inițială de a rula algoritmi concurenți folosind multi-threading (`std::async`) a dus la rezultate eronate, deoarece funcțiile de sistem de operare (ex. `GetProcessMemoryInfo`) returnau consumul total al

procesului, nu al fiecărui thread în parte. Problema a fost rezolvată prin abandonarea concurenței și executarea secvențială, izolând astfel perfect mediul pentru fiecare algoritm.

Directii viitoare: Ca propuneri pentru continuarea cercetării, se sugerează testarea algoritmilor pe arhitecturi paralele masive (GPGPU, folosind CUDA) sau analizarea performanței unor hibridi moderni adaptați la mărimea cache-ului, precum *TimSort*.

Bibliografie

- [1] Adriana Popovici, *Lucrare științifică: Structura unei lucrări științifice*, Metode și Practici în Informatică, Facultatea de Informatică, Universitatea de Vest din Timișoara.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

Anexe

- **Anexa A:** Codul sursă integral în limbajul C++17, disponibil în repository-ul GitHub atașat.
- **Anexa B:** Datele brute obținute din evaluări (fișierul `benchmark_results.csv`), incluzând toți timpii de execuție în nanosecunde și variația memoriei pentru configurațiile $N = 50$ și $N = 50.000$.